

Design Brief

DeskHive — Coworking Space Booking Web App

Phase 1 MVP — Updated v2 (Architecture-Aware)

Field	Value
For	Makhbuba (Designer)
From	Ikhtiyor (Business Analyst)
Project	DeskHive — Coworking Space Booking Web App
Phase	MVP (Phase 1)
Timeline	~2 weeks (foundation by end of Week 1, full delivery by end of Week 2)
Version	v2 — updated after technical architecture finalized

1. What We're Building

DeskHive is a web application that helps remote workers find and book a desk at a coworking space — think of it as Airbnb, but for individual workdays at coworking spaces instead of nights at apartments.

A user opens the site, browses coworking spaces in their city, picks a specific desk on a specific date, and books it. An admin on the other side approves the booking. The user shows up to work that day. That's the whole product.

This first version is a **functional MVP** — the goal is to prove the booking experience works end-to-end. We're keeping the feature set tight and the design clean so we can ship in 2 weeks.

2. Who It's For

Primary user — The Guest

- A remote worker, freelancer, or business traveler who needs a desk for the day.
- Tech-comfortable, used to apps like Airbnb, Booking.com, or Uber.
- Wants to find a place quickly, see clearly that they have a confirmed reservation, and not deal with friction.
- Mood: in-transit, often on the go, sometimes frustrated by hard-to-use sites.

Secondary user — The Admin

- Manages the coworking spaces and the bookings.
- Will spend more time in the system than guests will.
- Needs clarity and control more than visual delight.
- Mood: focused, doing repetitive tasks, wants efficiency.

3. Design Direction — Modern & Minimal

The Vibe

Think **Linear**, **Notion**, **Stripe**, **Vercel** — that family of products. Clean, confident, trust-inspiring. Generous whitespace. Excellent typography. Subtle, purposeful color. No decoration for decoration's sake.

This direction was chosen because:

- It signals trust and professionalism (important for a marketplace where money will eventually change hands).
- It's faster to design and build (we have 2 weeks).
- It ages well — it won't look dated in 6 months.
- It puts the content (spaces, desks, bookings) in the spotlight, not the chrome around it.

Reference Sites for Inspiration

- **linear.app** — typography, spacing, button styles
- **notion.so** — calm color palette, clear hierarchy
- **stripe.com** — trust signals, professional feel
- **vercel.com** — minimal but warm

What to AVOID

- Heavy gradients, drop shadows, glassmorphism — keep effects subtle.
- Stock illustrations of "diverse people working on laptops" — feels generic.
- Loud colors, busy backgrounds, animated hero sections.
- Marketing-heavy landing page — this is a utility, not a sales site.

4. The Screens You'll Be Designing

There are **8 screens** in this MVP. They split into three groups: public, guest-only, and admin-only.

PUBLIC (anyone can see these)

4.1 Landing / Browse Spaces (/)

The homepage. Shows a grid or list of all available coworking spaces. At the top: a simple search field where users can type a city name to filter the list. Each space card shows: an image, the space name, and the city.

- **Empty state:** "No spaces available yet."
- **Loading state:** Some indication content is loading.
- **Error state:** "Something went wrong. Please try again."

4.2 Space Detail (/spaces/:id)

When a user clicks on a space card, they land here. Shows:

- The space's primary image (larger)
- Name, city, address, full description
- **A date picker** — required before any booking can happen. Use the browser's native date picker (see Section 5.5).
- A list of **desks** available in that space — each with a label (e.g., "Desk-1"), a price per day, and an availability badge ("Available" or "Unavailable" for the chosen date)
- A "Book this desk" button on each desk (disabled until a date is selected, and disabled if the desk is unavailable)

4.3 Register (/register)

A simple sign-up form. Three fields: email, full name, password. One "Create account" button. Standard form validation states (inline errors).

4.4 Login (/login)

A simple login form. Two fields: email, password. One "Log in" button. Link to "Register" for new users.

GUEST-ONLY (must be logged in as a Guest)

4.5 My Bookings (/my-bookings)

A list of all the guest's bookings. Each booking row shows:

- The space name, desk label, and date
- A **status badge** (see Section 5 below for the four statuses)
- A "Cancel" button — but ONLY if the status is "Pending"

Empty state: "You haven't booked anything yet. Browse spaces to get started."

ADMIN-ONLY (must be logged in as Super Admin)

4.6 Admin Spaces List (/admin/spaces)

A table or list of all coworking spaces. Each row shows space name, city, status. A "New Space" button at the top. Clicking a row goes to the edit screen (4.7).

4.7 Admin Space Edit (/admin/spaces/:id)

A form to edit a space's details (name, city, address, description, image URL). Below the form, a section for managing the desks inside that space — list of existing desks with edit/deactivate options, and an "Add Desk" button (label + price).

4.8 Admin Bookings List (/admin/bookings)

A table showing every booking on the platform. Each row shows: guest name, space name, desk label, date, status badge. For bookings in "Pending" status, two action buttons: **"Confirm"** (turns it green) and **"Reject"** (turns it red).

5. Design System — What You Need to Define

You don't need to build a full design system. You need just enough that the developer can apply your work consistently across all 8 screens. Specifically:

5.1 Color Palette

- 1 primary brand color (this is THE DeskHive color — used for primary buttons, key links, brand moments)
- 1 neutral palette (whites, grays, near-blacks for text, borders, backgrounds)
- 4 status colors for booking badges:
 - **Pending** (yellow/amber feel — warm, waiting)
 - **Confirmed** (green feel — success, secured)
 - **Rejected** (red feel — clear stop)
 - **Cancelled** (gray feel — neutral, terminated)

5.2 Typography

- 1 typeface for the whole product (we recommend a clean, modern sans like Inter, Geist, or similar)
- A type scale with about 4–6 sizes (heading large, heading medium, body, caption — that range)
- Clear weight rules (regular, medium, semibold)

5.3 Components

At minimum, design these core components in their default + interactive states. *Note: these are the actual component names already created in code — Section 5.5 explains how this connects.*

- **Button** (primary, secondary, disabled, loading) — used everywhere
- **Input field** (default, focused, with error message)
- **FormField** — wrapper around input that includes label + error text
- **Card** — used for space listings on the homepage
- **StatusBadge** — the four booking status badges (this is ONE component used everywhere — see Section 5.5)
- **DataView** — wrapper that handles the four UI states (loading/empty/error/loaded). You're designing the visual treatment for each of these four states.
- **Header** — site header with logo, nav, login/logout button

5.4 Spacing & Layout

- A consistent spacing scale (something like 4 / 8 / 16 / 24 / 32 / 48 px)
- Page maximum width and side padding for the main content area
- Header layout (logo, nav, login/logout button)

5.5 Technical Context You Should Know

The technical architecture has been finalized. This section gives you the few specific decisions that affect *what* you should design (not *how* you design — that's still your call). All of this is information you **can ignore if you prefer to work at the pure visual level** — your handoff still works. But knowing it can save you time.

Tech stack (for context)

- **Framework:** Next.js 16 with React
- **Styling:** Tailwind CSS v4 (note: v4, not v3 — slightly different syntax for the configuration file)
- **Component library:** None. Pure Tailwind utility classes on plain semantic HTML elements (real `<button>`, `<form>`, `<input>`, `<label>`, etc.)

How your design will be applied

Every visual decision you make lives in **ONE FILE** called `src/app/globals.css`, inside a Tailwind v4 `@theme` block. That's where colors, fonts, spacing tokens, and other design tokens live. The developer translates your design into that block; the components automatically pick up the tokens.

This means: **you don't need to specify CSS for each screen separately**. Define your tokens once, and they cascade through the entire app.

The StatusBadge is one component

The four booking status colors (Pending, Confirmed, Rejected, Cancelled) are rendered by a single `<StatusBadge>` component used across all screens. Design the four variants once. They'll appear correctly everywhere automatically.

Date picker is native — don't design one

Important: the date picker is the browser's native `<input type="date">`. This means you do **NOT** need to design a custom calendar widget. Each browser provides its own (Chrome, Safari, Firefox each look slightly different, and that's OK for the MVP). Save yourself the time.

DataView handles all four UI states

Every screen that loads data is wrapped in a `<DataView>` component that automatically handles the four states (loading, empty, error, loaded). You design the visual treatment for each of these four states once, and they apply to every data-loading screen consistently.

Optional: deliver design tokens as code if comfortable

If you're comfortable working in code (e.g., via Claude Code or Claude Artifacts), you can deliver your design tokens directly as a Tailwind v4 `@theme` CSS block. This is the fastest possible handoff because no translation is needed. If you'd rather work in Figma or another visual tool — also fine. Both paths work.

In short: Your design freedom is high (no component library imposing defaults) but your design is also concentrated in one CSS file. Three things you don't have to design: a date picker, separate status badges per screen, or per-screen state handling — these are centralized.

6. Important UX Rules to Reflect in Your Design

These aren't just guidelines — they're the contract the engineering side has committed to:

- **Booking confidence is the #1 emotional requirement.** A guest should never have to wonder if their booking is real. The status badge needs to be unambiguous and visually dominant on the My Bookings page.
- **The date picker on the Space Detail page is the moment of truth.** The interface should feel obvious: pick a date, see what's available, click to book. Don't bury the date picker.

- **Disabled states must look obviously disabled.** Buttons that can't be clicked yet (e.g., "Book this desk" before a date is chosen) need to LOOK unclickable. Don't trick the user.
- **Empty states matter.** Users will see them often early in the product's life. Don't make them feel like errors.
- **Forms are simple.** No multi-step wizards, no fancy animations between fields. Inline validation errors below each field.
- **No modal dialogs in the booking flow.** When a booking succeeds, the user is taken to the My Bookings page. Don't pop up a "Success!" modal.
- **No custom interactive widgets.** If something would normally need a custom dropdown, custom autocomplete, or modal-style popover — please flag it instead. The architecture uses native HTML semantics for accessibility.

7. What's Out of Scope (Don't Design These)

To keep us honest about the 2-week timeline, please **do NOT design** any of the following — they're explicitly Phase 2 and would just be wasted work:

- Forgot password / password reset screens
- User profile / account settings pages
- Email templates or notification UI
- Photo upload interfaces (for now, photos are just URLs the admin types in)
- Reviews, ratings, or comments anywhere
- Messaging between users
- Map-based search or filters by amenities
- Payment / checkout / credit card screens
- A "Pricing" or "About" or "Marketing" page
- Mobile app screens (the website should work in mobile browsers, but we're not designing native app UI)
- Meeting room booking screens (Phase 1 is desks only)
- Multi-language toggles
- Analytics dashboards
- Custom date pickers (use the browser native — see Section 5.5)
- Custom dropdowns, modals, or popovers (architecture uses semantic HTML only)

If something feels like a "nice to have" — it's almost certainly Phase 2. Ask before designing it.

8. Working in Parallel With the Build

Here's how the workflow looks across the 2 weeks:

- **Week 1, Days 1–3:** The development side is setting up infrastructure and building the first screens with plain default styling (no design applied). You can be working on the design system foundation during this time — colors, typography, core components.
- **Week 1, Days 4–5:** Hand off the foundation (colors, type, button styles, status badges) so the developer can start applying it as new screens are built. You begin designing the full screens.

- **Week 2:** You deliver screen designs as they're ready. The developer reskins each screen as your design lands. By end of Week 2, the design and the build merge into the final product.

This means you don't need to wait until everything is built to start shipping work, and the developer doesn't need to wait for all your designs before they can build. It's the most efficient setup for our timeline.

9. Deliverables

Whatever tool you prefer is fine — Figma, HTML/Tailwind components, or AI-generated designs in Claude. What we need at the end is:

Required

- Designs for all 8 screens (Section 4) in their default loaded state
- The design system foundation (Section 5) — colors, type scale, core components
- **StatusBadge designs** — the 4 booking statuses, designed once (one component used everywhere)
- **DataView state designs** — visual treatment for the loading, empty, error, and loaded states (one component used by every data-loading screen)

Strongly preferred (but not blocking)

- Designs for form validation states (inline errors)
- Mobile/responsive layout hints — not full mobile designs, but a note on how each screen should adapt at narrow widths
- Hover and focus states for interactive elements
- **Optional but ideal:** a Tailwind v4 @theme CSS block with your design tokens (colors, fonts, spacing). This is the fastest handoff path — see Section 5.5.

Not needed

- Multiple visual concepts/options — pick a direction and execute, don't present 3 options.
- Full mobile designs (we're desktop-first for MVP).
- Animations or motion design (defaults are fine).
- Brand identity work beyond the primary color (logos, illustrations, etc. — Phase 2).
- A custom date picker (browser native is being used).

10. Practical Notes

- **Tool of your choice.** Use whichever tool helps you work fastest. If you want to use Claude or Claude Code to generate components — great, the output will integrate cleanly with our build.
- **Default to defaults.** If you're unsure whether to design something custom or use a sensible default, default. Time is the constraint.

- **One question at a time.** When you have questions or want feedback, send them in batches (rather than one at a time over a day) so I can respond efficiently between my orchestration sessions on the build side.
- **Open communication.** If something in this brief seems wrong, missing, or worth challenging — please push back. You're the design expert; I'm the BA. The brief is a starting point, not a contract.
- **Architecture document available on request.** If you want full technical context (especially the file paths, component names, and the exact CSS file where your design tokens will live), I can share the architecture document our AI architect produced. It's about 20 pages of technical detail — read only what's useful to you.

11. Quick Reference — Project Facts

Fact	Value
Product name	DeskHive
Product type	Web application (desktop-first; works in mobile browsers)
Framework	Next.js 16 with React
Styling system	Tailwind CSS v4 (note: v4, not v3)
Component library	None — pure Tailwind utility classes on semantic HTML
Design tokens location	src/app/globals.css inside a @theme block
Date picker	Browser native <code><input type="date"></code> — no custom design needed
Number of screens	8
Number of user roles	2 (Guest, Super Admin)
Currency for prices	USD
Languages	English only
Booking unit	One full calendar day per desk (no hourly bookings in MVP)
Design timeline	Foundation by end of Week 1; full delivery by end of Week 2
Build timeline	2 weeks total; design and build happen in parallel

Looking forward to seeing what you create, Makhbuba. Let me know if anything in this brief is unclear or worth discussing before you dive in. — Ikhtiyor